Comprehensive Project Walk-through of the Bookory MVC Application

This document is a complete, **step-by-step explanation** of the Bookory e-commerce website. The aim is to explain each folder and file in a **baby-feeding manner**—plain language that introduces concepts such as models, controllers, views, services, enumerations, ViewBag, ViewData, TempData, and Razor syntax like @model IEnumerable<T>. At the end you should understand the **workflow** for each module (e.g., browsing books, adding to cart, checkout, order processing, admin dashboard) and how the pieces connect.

---

1. High-level overview

The Bookory project is a **full-stack web application** built with **ASP.NET Core MVC** and **Entity Framework Core**. It mimics a small online bookshop. Users can register, log in, browse books by category, put books in a shopping cart or wishlist, place orders, pay via UPI or cash-on-delivery, view their order history, and request returns. Administrators have extra powers: they can add/edit/delete books, see all orders and sales statistics, and view all users.

MVC stands for Model-View-Controller—three roles in the app:

- **Models** represent the data (e.g., Book, Order, User). They map to database tables and include validation rules.
- **Controllers** handle HTTP requests. They decide what to do when someone visits a page or submits a form. They coordinate between models and views.
- **Views** are Razor (.cshtml) templates that generate HTML sent to the browser. They show data from the controller and use loops, conditions and tag helpers.

The project also uses **services** to encapsulate business logic (e.g., creating orders or payments) so controllers stay simple. **Dependency injection (DI)** provides these services and the database context to controllers.

---

2. Folder structure

When you unzip Bookory-DEV1 (2).zip you see a Bookory-DEV1 folder. Inside are these key directories:

2.1 Root files

File	What it does	Baby-feeding explanation
BookStoreMVC.sln	Solution file for Visual Studio	Think of it as the project wrapper; it tells Visual Studio which projects to build.
BookStoreMVC.csproj	C# project file	Lists NuGet packages (e.g., Entity Framework,

File	What it does	Baby-feeding explanation
		Microsoft.AspNetCore) and configuration (target .NET 8). Without this file the compiler does not know how to build the project.
Program.cs	Application startup	This is where the web host is configured. It **registers services** (controllers, database, authentication, custom services) and sets up routing. It also applies database migrations and seeds sample data via DbInitializer.
appsettings.json	Configuration	Contains the database connection string (DefaultConnection) and other settings. Environment-specific overrides can go in appsettings.Development.json (not included here).
wwwroot	Static assets	Holds css/site.css, images (book pictures, background images) and other files served directly by the server. These are the files that your browser downloads (like images and CSS).
Views/ _ViewImports.cshtml	Razor directives	Imports namespaces and tag helpers so you don't have to write @using Microsoft.AspNetCore.Identity at the top of every view.

2.2 Models folder

The Models folder contains **data classes** and a custom DbContext:

2.2.1 ApplicationDbContext.cs

- Inherits from DbContext. This class tells Entity Framework which tables to create. Each DbSet<T> corresponds to a table (e.g., Books, Orders, Users).

- It configures relationships using the fluent API: e.g., a CartItem has one User and one Book, an OrderItem belongs to one Order and one Book, and PaymentStatus enums are stored as strings for readability.
- It includes a method OnModelCreating to set up cascade deletes: if a book or user is deleted, related cart items/orders are removed automatically.

2.2.2 Entity classes

Each class in the Models folder represents a table with columns and navigation properties:

Class	Purpose	Important fields	Notes
User	Represents a customer or admin	Username, PasswordHash, Role (see enum), Email, FirstName, LastName, DateOfBirth, Gender, Phone, Address	UserRole enum distinguishes CUSTOMER vs ADMIN. Navigation: CartItems, Orders. Password is stored securely as a **hash** plus salt (split by a period).
Category	Book category (Fiction, Science…)	Name	Books navigation lists all books in this category.
Book	Product for sale	Title, Description, Price (decimal for money), CategoryId, StockQuantity, ImageUrl	Navigation: Category, CartItems, OrderItems. ImageUrl defaults to a placeholder; when an admin uploads an image it is saved in /images and the path is set here.
CartItem	One row in a user's shopping cart	UserId, BookId, Quantity	Navigation: User, Book.
Order	A customer's order	UserId, TotalAmount, OrderDate, Status (see OrderStatus enum), shipping details (name, address, city, state, zip, phone)	Navigation: collection of OrderItems, optional Payment. The Status can be PENDING, SHIPPED, DELIVERED, CANCELLED, plus return states (see below).

Class	Purpose	Important fields	Notes
OrderItem	A line item in an order	OrderId, BookId, Quantity, UnitPrice	Navigation: Order, Book. UnitPrice stores the price at the time of order.
Payment	Payment record for an order	OrderId, Amount, PaymentMethod (e.g. UPI or COD), PaymentStatus (enum), PaymentDate	Navigation: Order. The PaymentStatus can be PENDING, COMPLETED, or FAILED.
WishlistItem	A user's saved item	UserId, BookId	Navigation: User, Book. The file name has two dots due to a naming typo (WishlistItem..cs) but the class is correct.
PaginatedList<T>	Helper for server-side paging	Properties: PageIndex, TotalPages	Provides HasPreviousPage/HasNextPage and a CreateAsync method that uses Skip and Take to fetch one page from a query.
DbInitializer	Seeds sample data	Populates categories (Fiction, Science, Technology…), sample books (e.g., *The Great Gatsby*), and an admin user. It uses HashPassword to securely store the admin's password. Called in Program.cs during startup.	

2.2.3 Enumerations (enums)

In C#, an **enum** defines a set of named integral constants. Enums make code more readable by giving meaningful names instead of magic numbers. In this project:

- UserRole { CUSTOMER, ADMIN } – distinguishes whether a user is a normal shopper or an administrator. Controllers check User.IsInRole("ADMIN") to grant admin privileges.
- Gender { MALE, FEMALE, OTHER } – used in the User model for profile information.
- OrderStatus { PENDING, SHIPPED, DELIVERED, CANCELLED, RETURN_REQUESTED, RETURN_SHIPPED, RETURN_DELIVERED } – tracks where an order is in its lifecycle: just placed, shipped, delivered, cancelled, or in various return stages. This enum prevents invalid strings from being stored and simplifies status comparisons.
- PaymentStatus { PENDING, COMPLETED, FAILED } – indicates whether a payment is waiting, completed successfully, or failed (e.g., an online payment error). For COD orders the status stays PENDING until the order is delivered.

---

3. Services

Services live in the Services folder. They encapsulate business rules and data access so controllers can stay thin. Each service has an **interface** (the contract) and a **class** (the implementation). Dependency injection will provide these to the controllers.

Service	Interface & methods	Implementation summary	Baby-feeding view
BookService	IBookService – GetCategoriesAsync(), GetBooksAsync(int? categoryId), GetByIdAsync(id), AddAsync, UpdateAsync, DeleteAsync	Uses ApplicationDbContext to fetch categories and books, optionally filtering by category. Adds new books, updates details or image, and deletes a book.	Imagine a library manager who knows all the genres (categories) and books. You can ask for all books, books in a genre, details of one book or instruct the manager to add/edit/remove a book.
CartService	ICartService – GetCartItemsAsync(userId), AddToCartAsync(userId, bookId), UpdateQuantityAsync(userId, cartItemId, qty), RemoveAsync(userId, cartItemId), ClearAsync(userId)	Fetches cart items with book and category details for display. Adds a book to cart (if stock > 0), increases quantity without exceeding stock, removes a row, or clears the cart.	Think of this as a shopping cart helper who handles your items. You tell them to add or remove books, update quantity, or empty the cart.

Service	Interface & methods	Implementation summary	Baby-feeding view
OrderService	IOrderService – GetOrdersAsync(userId, isAdmin), GetOrderAsync(orderId), CreateOrderAsync(userId, CheckoutVm), UpdateStatusAsync(orderId, status)	Provides order lists: if isAdmin=true returns all orders; otherwise returns only the user's. When creating an order it runs inside a database **transaction** to ensure stock updates and cart clear happen atomically. UpdateStatusAsync lets admin mark an order as delivered (completing payment) or other statuses.	Acts like the system that turns your cart into a final order: records items, total, shipping info, deducts stock, clears the cart and handles cancellations.
PaymentService	IPaymentService – ProcessPaymentAsync(userId, orderId, amount, method, success)	Validates that the order belongs to the user, then creates a payment record. If the method is online and success=true, sets PaymentStatus=COMPLETED and keeps the order pending for shipping; if failed, cancels the order. For cash-on-delivery it sets PaymentStatus=PENDING until the admin marks the order delivered.	A simple payment processor: it knows if you paid online or COD and sets the payment and order status accordingly.
WishlistService	IWishlistService – GetWishlistItemsAsync(userId), AddToWishlistAsync(userId, bookId), RemoveFromWishli	Retrieves the user's wishlist items including book and category details. Adds a new entry if it doesn't already exist. Removes an entry if	Think of this as a "save for later" list manager. It keeps track of your saved books and prevents duplicates.

Service	Interface & methods	Implementation summary	Baby-feeding view
	stAsync(userId, wishlistItemId)	it belongs to the user.	

4. Controllers and their workflows

Controllers live in the Controllers folder. They each inherit from Controller and have methods (actions) corresponding to URL routes. Controllers use services and the ApplicationDbContext to perform tasks. Below we explain each controller and the flow of requests.

4.1 AccountController – authentication and user profiles

- **Register (GET/POST)** – Shows the registration form and creates a new user. The POST action validates that all fields are filled, passwords match, and the username is unique. It converts the gender string to the Gender enum and hashes the password using PBKDF2. After saving the user to the database, it automatically signs them in using SignInUser(user) (creates authentication cookies) and redirects to the home page.
- **Login (GET/POST)** – Shows the login form and validates credentials. VerifyPassword splits the stored PasswordHash into salt and hash, then computes the hash of the entered password. If correct, it creates claims: NameIdentifier (user ID), Name (username), Role (enum converted to string) and signs in the user. If the user is an admin, they are redirected to the admin dashboard; otherwise to the home page.
- **Logout** – Calls SignOutAsync to remove the authentication cookie and returns to the home page.
- **ForgotPassword** – A placeholder action that simply displays a message; in a real app it would send a password reset email.
- **EditProfile / Profile** – Allows a logged-in user to view and update personal details except for username, role and password. The controller removes those fields from model validation to prevent them from being overwritten.

4.2 AdminController – admin dashboard and lists

- All actions are guarded by [Authorize(Roles = "ADMIN")] to restrict them to administrators.
- **Dashboard** – Gathers summary metrics for display: total users, total books, total orders, total sales (sum of completed payments minus cancellations), count of in-stock and out-of-stock books. It also loads the five most recent orders with user, payment and book details to show a list of latest transactions.
- **Users** – Returns a list of all customers (not admins) so the admin can see who is registered.
- **Instock / Outofstock** – Returns lists of books where StockQuantity > 0 or == 0 respectively. Useful for inventory management.

4.3 BooksController – browsing and admin book management

- **Constructor** – Injects the IWebHostEnvironment to get the path to wwwroot (for saving uploaded images) and IBookService for data access.
- **Index** – Supports optional category filtering and server-side pagination. It loads categories for a filter sidebar and constructs a LINQ query on books. If a category is selected, it filters the query. It then calculates the page number and size, and uses PaginatedList.CreateAsync to fetch only the current page. The paginated list is passed to the view which uses @model PaginatedList<Book>.
- **Details** – Loads a single book with its category and shows details plus an Add to Cart button.
- **Create (GET/POST)** – Admin-only. The GET action loads categories for a dropdown. The POST action accepts the bound Book (title, description, price, category, stock) and an uploaded image file (IFormFile file). If a file is present, it generates a unique filename (using Guid.NewGuid()), saves the file to wwwroot/images and sets ImageUrl accordingly; otherwise uses a default. Then it calls IBookService.AddAsync to insert the record.
- **Edit (GET/POST)** – Admin-only. GET loads the book and categories; POST updates the record. If a new image is uploaded, it deletes the old file and saves the new one, updating ImageUrl.
- **Delete (GET/POST)** – Admin-only. GET shows the details of the book to confirm deletion; POST finds the book, removes its OrderItems (to avoid foreign key constraint errors) and then deletes the book.

4.4 CartController – cart operations and checkout

- The entire controller is protected by [Authorize]; only logged-in users can have a cart.
- **Index** – Calls ICartService.GetCartItemsAsync(CurrentUserId) to get all items in the user's cart along with book and category info, then passes them to Views/Cart/Index.cshtml for display.
- **Add** – Adds a book to the cart (if stock > 0) via AddToCartAsync. It accepts goToCart flag; if true it redirects to /Cart, otherwise it goes back to the book details page.
- **Update** – Changes the quantity of a cart item. The service clamps quantity between 1 and available stock.
- **Remove** – Removes a cart item.
- **Checkout (GET)** – Loads cart items; if empty, redirects to cart. Computes Subtotal (sum of price × quantity), Shipping (0 by default) and Total (subtotal + shipping) and stores them in ViewBag. It then displays the checkout form (full name, address, city, state, zip, phone) bound to a CheckoutVm.
- **Checkout (POST)** – Accepts a CheckoutVm and validates required fields. Calls OrderService.CreateOrderAsync to convert cart items into an Order and OrderItems. Clears the cart and deducts stock. Redirects to payment.
- **Payment (GET)** – Loads the order by orderId. Checks that the order belongs to the current user. Computes totals again and constructs a PaymentVm with default values (method = "UPI", success = true). Shows a form to simulate payment or choose COD.

- **Payment (POST)** – Validates the model then calls PaymentService.ProcessPaymentAsync. For online payments it sets status based on the Success flag; for COD it keeps payment PENDING. Redirects to Success with ok parameter indicating whether the payment succeeded.
- **CodSuccess** – Shortcut for cash-on-delivery: processes the payment with method "COD" and marks success = true. Payment status remains PENDING until the admin delivers the order.
- **Success** – Shows the order summary and whether the payment succeeded. The view can display a message or allow the user to see their order details.

4.5 OrdersController – order history and admin status updates

- Protected by [Authorize] to ensure only logged-in users access it.
- **Index** – Loads orders. If the user is an admin (checked with User.IsInRole("ADMIN")), it loads *all* orders with related User, Payment, and OrderItems → Book. Otherwise it filters to only orders where UserId == current user. Orders are sorted by OrderDate descending and passed to the view.
- **Details** – Shows all details for a single order: its items with book and category info, the user (for admin view), and payment. If a customer tries to view someone else's order (i.e., not admin and UserId != current user), it returns 403 (Forbidden).
- **UpdateStatus (POST, Admin)** – Allows an admin to change an order's status via form submission. It loads the order with Payment and OrderItems → Book. It stores the old status, sets the new status, and applies side effects:
  - If status becomes DELIVERED, it sets Payment.PaymentStatus = COMPLETED.
  - If status becomes CANCELLED and it was not already CANCELLED, it loops through OrderItems and returns the quantity to each Book.StockQuantity. This ensures stock is replenished when an order is cancelled after creation. After updating, it saves changes and redirects back to the order details page.
- **RequestReturn (POST, Customer)** – Allows a customer to request a return on a delivered order. It loads the order by id and user, checks the order is delivered, sets status to RETURN_REQUESTED and saves.
- **Cancel (POST, Customer)** – Allows a customer to cancel an order that is not yet delivered or in a return state. It includes OrderItems → Book so it can return stock. If the order is in a forbidden state (delivered or already cancelled/returning), it just shows details. Otherwise it loops over OrderItems and increases each Book.StockQuantity by Quantity, sets status to CANCELLED and saves.

4.6 WishlistController – saved items

- Entire controller is [Authorize] so only logged-in users can save items.
- **Index** – Calls WishlistService.GetWishlistItemsAsync to get the current user's wishlist including book and category details. The view displays them in a list.
- **Add (POST)** – Adds a book to the wishlist via WishlistService.AddToWishlistAsync. The service checks if it already exists to prevent duplicates. Stores a message in

TempData["Message"] to show feedback on the next page and redirects back to the books page.

- **Remove (POST)** – Removes a wishlist item using WishlistService.RemoveFromWishlistAsync, again storing a message in TempData and redirecting to the wishlist page.

---

5. Views and Razor concepts

Views are .cshtml files under the Views folder. They contain **HTML** with inline **Razor** syntax. Razor begins with @ and allows you to embed C# expressions or statements in HTML. Here is how views work and some key concepts:

5.1 View structure

Each view often begins with @model to declare the type of data it expects. For example:

@model IEnumerable<BookStoreMVC.Models.Book>

This tells Razor that the page's model is a collection of Book objects. When you write @foreach (var book in Model) inside the view, it loops over this list. If you don't specify @model, the page will use dynamic data (ViewBag or ViewData) instead.

Views also use the **Layout system**. The file Views/Shared/_Layout.cshtml defines the page skeleton (header, navbar, footer). Each view can set ViewData["Title"] to change the <title> tag or show a page heading. @RenderBody() in the layout inserts the view's content. @RenderSection() is used for optional sections like extra JavaScript.

5.2 Important Razor features explained

- **@model** – Declares the type of the page model. The model can be a simple type (e.g., string), a complex type (e.g., User), or a list (e.g., IEnumerable<Book>). This makes the view strongly typed and enables IntelliSense in editors.

- **@foreach** – Loops over lists to generate repeated HTML (e.g., display each book in a grid). Example:

```
@foreach (var item in Model) {
   <div>@item.Title - @item.Price</div>
}
```

- **@if / @else** – Conditional logic to show or hide parts of the page. Example: show an "Add to Cart" button only if book.StockQuantity > 0.

- **Tag Helpers (asp- attributes)** – Provide strongly typed, compile-time checked HTML. Examples:

  - <form asp-controller="Account" asp-action="Login"> automatically sets the form's action URL.

  - <a asp-controller="Books" asp-action="Details" asp-route-id="@book.Id">Details generates a link with the appropriate route.
  - <input asp-for="Email" class="form-control" /> binds the input to the Email property of the model and generates appropriate id and name attributes.
- **ViewBag** and **ViewData** – Temporary dictionaries for passing values from controller to view. ViewBag is dynamic (you can write ViewBag.Total = 100 and read it in the view) while ViewData is a string-keyed dictionary (ViewData["Title"] = "Home"). Both last only for the current request.
- **TempData** – A dictionary that survives one redirect. Useful for passing success/error messages after a POST. Example: in WishlistController, after adding an item you set TempData["Message"] = "Book added!"; the next page can display it.
- **Validation helpers** – Tag Helpers like and show model validation errors next to inputs. They are automatically filled by MVC when ModelState.IsValid is false.
- **Sections** – @section Scripts { ... } in a view can inject extra JavaScript into the layout's @RenderSection("Scripts", required: false) placeholder.

5.3 Views by folder

Below is a brief description of each view file and what it does. They all use Bootstrap classes (e.g., row, col, btn, card) for styling.

5.3.1 Views/Home

- **Index.cshtml** – Landing page. Shows a hero banner with a call-to-action and a grid of featured books (first 8 books from the database). Loops over the IEnumerable<Book> model and for each book displays its image, title, truncated description, price, category badge, and buttons to view details or add to cart.
- **AboutUs.cshtml** and **ContactUs.cshtml** – Static pages about the company and contact form. They use the layout and provide content sections. There is no model here; information can be hard-coded or passed via ViewBag.
- **Privacy.cshtml** – Standard privacy policy page; generated by default in ASP.NET templates.
- **Error.cshtml** – Shown when an unhandled exception occurs. It reads error details from IExceptionHandlerPathFeature and shows the stack trace if in development mode.

5.3.2 Views/Account

- **Register.cshtml** – Shows the registration form. Uses <form asp-action="Register"> to post back to AccountController.Register (POST). Inputs are bound with asp-for to properties like Username, Email, Password, ConfirmPassword, FirstName, LastName, DateOfBirth, Gender, Phone, Address. Validation spans display errors (e.g., "Password required").

- **Login.cshtml** – Shows the login form. On invalid login, the controller adds an error to ModelState, and the view shows a message.
- **Profile.cshtml** – Shows the current user's profile data. Displays fields read-only and may include an edit button.
- **ForgotPassword.cshtml** and **AccessDenied.cshtml** – Simple pages: the former instructs you to check your email; the latter informs you that you don't have permissions.

5.3.3 Views/Books

- **Index.cshtml** – The main shop page. Accepts a PaginatedList<Book> model. It displays a sidebar of categories (from ViewBag.Categories) and a grid of books (current page only). At the bottom it shows pagination controls: previous/next page links with appropriate asp-route parameters to keep the category filter.
- **Details.cshtml** – Displays a single book's image, title, price, description and category. Provides an "Add to Cart" button. If stock is zero it shows "Out of stock". Also includes an "Add to Wishlist" form that posts to WishlistController.Add.
- **Create.cshtml** and **Edit.cshtml** – Admin pages with forms for entering or editing book details. The form uses asp-for for each property and a file input for the image. On edit, the current image is shown and a new file can replace it.
- **Delete.cshtml** – Shows a confirmation page before deleting a book. It displays the book details so the admin can verify they are deleting the correct item.

5.3.4 Views/Cart

- **Index.cshtml** – Shows all items in your cart. Loops through each CartItem to display book details, quantity and total per item. Provides buttons for increasing/decreasing quantity (asp-action="Update" asp-route-id="@item.Id" asp-route-qty="@item.Quantity + 1"), removing items, and proceeding to checkout. At the bottom it shows the subtotal, shipping, total and a "Checkout" button.
- **Checkout.cshtml** – Shows a summary of items and a form for shipping details. The form binds to CheckoutVm. Displays ViewBag.Subtotal, ViewBag.Shipping, ViewBag.Total. After submitting, the user is redirected to the payment page.
- **Payment.cshtml** – Presents payment options: UPI or credit card (fields for UPI ID, card number, expiry, CVV, card name). Uses PaymentVm as the model. On form submission it posts to CartController.Payment (POST). Offers a link to "Pay on Delivery" that calls CodSuccess.
- **Success.cshtml** – Shows a thank-you message and summarises the order. The model is the Order entity; it displays order number, date, total, status, and each ordered item. It can also display whether payment was successful using ViewBag.Ok.

5.3.5 Views/Orders

- **Index.cshtml** – Lists orders. The model is IEnumerable<Order>. For each order it displays order number, date, status, total amount, payment status and number of items. If the user is an admin, it may also show the customer's name. It includes buttons/links

to view details and, for admins, to change status (e.g., drop-down to mark as delivered or cancelled).

- **Details.cshtml** – Shows everything about one order: shipping address, user details (admin only), ordered items with unit price and quantity, payment method and status, and a timeline of status changes. If the viewer is the customer, it displays buttons to **Cancel** (calls OrdersController.Cancel) or **Request Return** (calls OrdersController.RequestReturn). If the viewer is the admin, it shows a form with a drop-down for OrderStatus (e.g., PENDING, SHIPPED, DELIVERED, CANCELLED, RETURN_REQUESTED), which posts to OrdersController.UpdateStatus.

5.3.6 Views/Admin

- **Dashboard.cshtml** – Shows summary cards for total users, books, orders, sales, out-of-stock and in-stock books. Below, it lists recent orders with details and statuses. Uses ViewBag values to display the metrics.
- **Users.cshtml** – Lists all customers with their usernames and emails. The admin can use this for quick customer support.
- **Instock.cshtml** and **Outofstock.cshtml** – Lists books based on stock quantity. The admin can see which books need restocking.

5.3.7 Views/Wishlist

- **Index.cshtml** – Shows the user's saved books. Loops over the WishlistItem model to display book images and titles with buttons to remove them or visit their details. Displays success messages from TempData["Message"] when a book is added or removed.

5.3.8 Views/Shared

- **_Layout.cshtml** – The master template. Includes the <head> (linking Bootstrap CSS and your custom CSS), a responsive navbar that shows "Home", "Shop", "Cart" with item count, "Wishlist", user profile dropdown and admin menu if the user is an admin. Defines @RenderBody() for content and @RenderSection("Scripts", required: false) for per-page scripts. The footer contains copyright text.
- **_AdminLayout.cshtml** – A variant layout used for admin pages. Has a sidebar menu linking to "Dashboard", "Users", "In-Stock", "Out-of-Stock", etc.
- **_ValidationScriptsPartial.cshtml** – Includes jQuery validation scripts for client-side form validation.

---

6. Special concepts explained

6.1 ViewBag vs ViewData vs TempData

- **ViewBag** – A dynamic property bag. It is actually a wrapper around ViewData that uses dynamic properties, e.g., ViewBag.TotalUsers = 5. Use when passing small pieces of data (numbers, strings) to a view for the current request only.

- **ViewData** – A dictionary (Dictionary<string, object>). Use ViewData["Title"] = "Books". It also lasts only for the current request. Both ViewBag and ViewData are good for small pieces of data; for complex models use the @model directive instead.
- **TempData** – Persists for one redirect. It uses session under the hood. Good for storing a success/error message after a POST/Redirect/GET pattern (e.g., after adding an item to wishlist). Once read, it is removed from storage (like a one-time note).

6.2 Enumerables in Razor: @model IEnumerable<T>

When a view declares @model IEnumerable<Book>, it means the page's model is a **collection** (list) of Book objects. This is common for list pages like Index.cshtml in the Books folder. Inside the view you loop over it:

```
@foreach (var book in Model) {
   <div>@book.Title – @book.Price</div>
}
```

If instead the model is a **single object** (e.g., @model Book), you access its properties directly (Model.Title). For forms that return multiple objects (e.g., Cart/Checkout), you use a custom **view model** class (e.g., CheckoutVm) with only the fields you need.

---

7. End-to-end workflows

Below are common user flows to illustrate how controllers and views work together.

7.1 Browsing and adding to cart

1. **User visits /Books** → BooksController.Index loads categories and a paginated list of books. The view loops through Model (the page of books) and shows cards.
2. **User clicks "Add to Cart"** on a book → link to CartController.Add(bookId, goToCart=true). The controller calls CartService.AddToCartAsync(CurrentUserId, bookId) and redirects to the cart page.
3. **Cart page** (Cart/Index.cshtml) shows each CartItem with quantity controls and a remove button. It calculates subtotals and total.
4. **User clicks "Checkout"** → CartController.Checkout (GET) displays the checkout form. The user enters shipping details.
5. **User submits the form** → CartController.Checkout (POST) converts the cart to an order via OrderService.CreateOrderAsync, clears the cart, and redirects to the payment page.
6. **Payment** → CartController.Payment shows amounts and payment method options. On submission, it calls PaymentService.ProcessPaymentAsync and redirects to the success page.

7.2 Wishlist

1. On a book details page, the user clicks **"Add to Wishlist"** (a small form) → posts to WishlistController.Add with the book ID. The controller calls WishlistService.AddToWishlistAsync to insert a row. Sets TempData["Message"] = "Book added to your wishlist!" and redirects back to books.
2. The books page reads TempData["Message"] and shows a notification.
3. The user visits /Wishlist → WishlistController.Index loads saved items and displays them in a list. They can click **"Remove"** which posts to WishlistController.Remove, deletes the row and shows a "Book removed" message.

7.3 Order management (admin vs customer)

1. **Customer** goes to /Orders → OrdersController.Index loads only their orders. They see a list with statuses and links to details.
2. **Admin** goes to /Orders → sees **all** orders. Each row shows user name, payment status, etc.
3. **Admin** clicks "View Details" on an order → OrdersController.Details shows full info plus a drop-down to update status. They can choose SHIPPED, DELIVERED, CANCELLED, etc. The form posts to UpdateStatus which applies side effects (complete payment or restock) and redirects back.
4. **Customer** can click "Cancel" or "Request Return" on their order details page. Both actions call POST methods (Cancel or RequestReturn) to update the OrderStatus and adjust stock accordingly.

7.4 Admin inventory management

1. **Admin** visits /Admin/Dashboard → sees summary cards and recent orders. This helps them track the business.
2. They can view **Users** to see a list of all customers.
3. They can manage books through /Books/Create, /Books/Edit, /Books/Delete. Creating or editing will upload images to wwwroot/images and set ImageUrl. Deleting will remove the record and associated OrderItems.
4. **Instock** and **Outofstock** pages show which books need restocking.

---

8. Conclusion

The Bookory MVC project is an instructive example of building a full-featured web application with ASP.NET Core. By separating concerns into models, controllers, services, and views, the application stays organised and easy to understand. Entities represent data with validation and relationships; controllers coordinate user requests; services encapsulate business rules like order creation, payment processing, cart management and wishlist logic; and views display information with Razor syntax and Bootstrap styling.

Understanding enums, ViewBag/ViewData/TempData, the @model directive and enumerables is key to working with Razor pages. The workflows described above demonstrate

how users and admins interact with the site from registration and browsing to checkout and order management. With this detailed, baby-feeding explanation you should now feel comfortable exploring the codebase and adapting it for your own learning or projects.